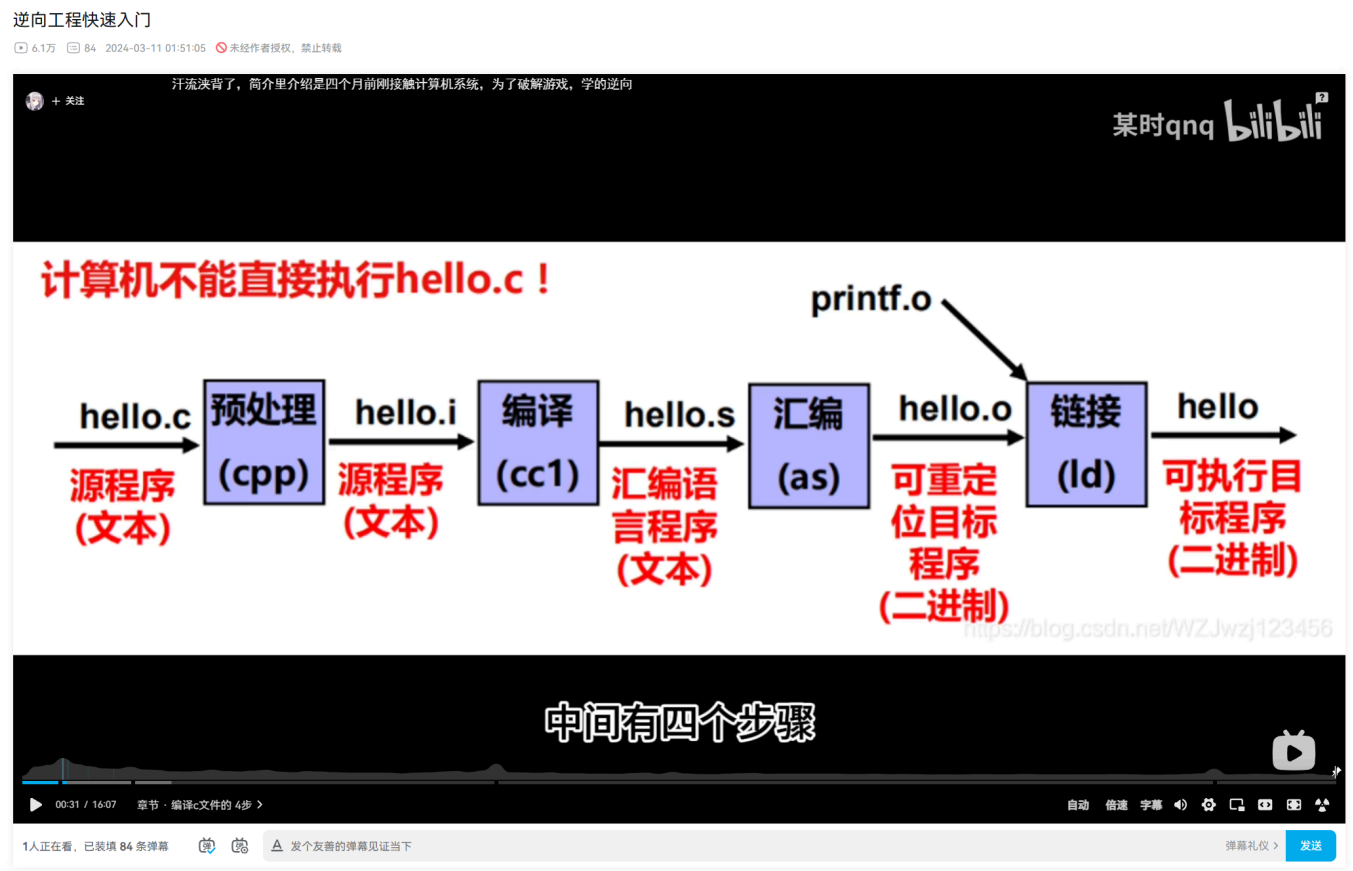




我两年前在b站做的一个视频里面，有这么一张图。

“编译”那个方块，其实是一个巨大的黑盒子。

一个语言，比如C，究竟是怎么从一个预处理过的本质上还是C的文件，变成汇编的呢？

这个视频，让我们来了解一下编译器的前端，后端，llvm, parser, lexer, ast, IR, aot和jit吧。

首先，我们要了解到，编程语言的光谱分为两端：compiled language（编译型语言）和interpreted language（解释型语言）

compiled language最终被编译成一个binary被执行，比如C，C++，Rust。

interpreted language则是某种中间形式被一个interpreter执行。比如Python。Python的CPython既是一个编译器也是一个解释器。

运行Python的时候，没有binary在你的disk上。而是CPython把.py编译成.pyc（Python bytecode），然后CPython又负责interpret这个bytecode，这一切都在内存里进行。

那你可能好奇，什么是bytecode？这个interpreter的解释也太模糊了。

那个视频看完，你会非常清晰理解CPython和编译器。

让我们现在跟着LLVM来了解C语言是怎么编译的。

一个编译阶段，分为前端、中端、和后端。前端包含lexer，parser，语义分析，和IR（intermediate representation，中间表示）生成。比如C和Rust可以被Clang和rustc分别编译成LLVM IR，一种长得有点像汇编的强类型低级语言。

中端则在IR上进行优化，比如耳熟能详的constant folding和dead code elimination等，缩减IR体量和提升内存效率等。后端负责把优化过的IR生成机器码（0和1）

我们先看一下编译器前端。

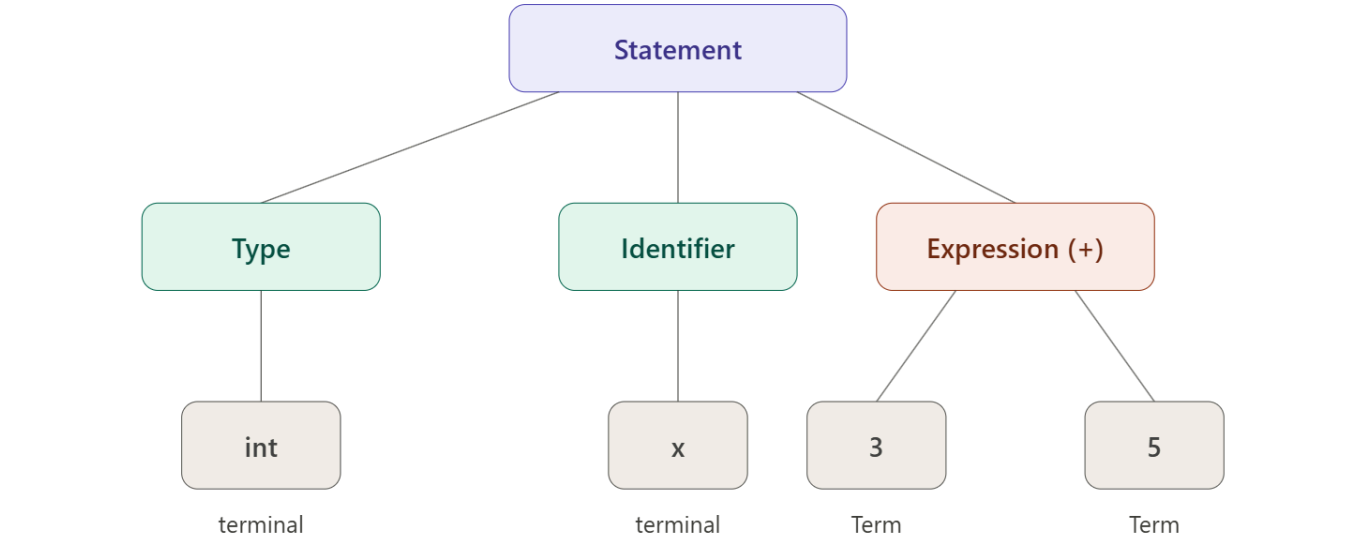
一个被预处理过的C文件，首先经过lexer变成token。int keyword是一个token，for也是一个token，变量名也是一个token。lexer可以用正则表达式工具（如flex）自动生成，也可以手写状态机。看到 [0-9]+ 就匹配成整数token，看到 [a-zA-Z_][a-zA-Z0-9_]* 就匹配成identifier（标识符）token。C的lexer如果看到 3abc，3 按整数规则匹配，结果下一个字符是 a，不符合任何整数的规则，lexer会报错。这个是compile time error。但是，如果一个数字overflow了，lexer不在乎，它只会把那个overflow的数字变成一个token。

parser做的事情就是把lexer吐出来的一堆token，组织成抽象语法树（Abstract Syntax Tree）。

parser 手里有一套语法规则（grammar），是编译器开发者定义的。比如CPython的grammar文件里就定义了Python所有的语法规则。

假设我们今天要编译 `int x = 3 + 5;`

lexer吐出来一串token：int, x, =, 3, +, 5, ; parser要搞清楚这些token之间的结构关系，最终组成这么一个树。



假设我今天设计了一个只能编译 `int x = 3 + 5;` 的编译器。为了编译 `int x = 3 + 5;`，我决定我的编译器有以下四条production rule（产生式）：

- `Statement` $\rightarrow$ `Type Identifier = Expression ;`
 - `Expression` $\rightarrow$ `Term + Term`
 - `Type` $\rightarrow$ `int`
 - `Term` $\rightarrow$ `number`
- 这里的符号分为 terminal（终结符）和 non-terminal（非终结符）。Terminal就是token，比如int，x，3。Non-terminal 就是 Statement、Expression，等，通过 production rule 展开成 terminal。

parser有两大类型：top-down和bottom-up。

top-down parser，是从最高层的production rule开始，逐步往下展开去匹配token。最有名的是LL parser。LL(1)就是每步只看一个token，LL(k)就是每步看k个token。比如我们的：`int x = 3 + 5;`。parser从Statement开始，偷看一个token，看到 `int`，展开 `Statement` $\rightarrow$ `Type Identifier = Expression ;`。然后进入Type，偷看还是 `int`，展开 `Type` $\rightarrow$ `int`，跟token匹配上了。接着期望Identifier，看到 `x`，match。期望=，看到=，match。现在该来Expression了，但parser不会直接去看token `3`。它先展开 `Expression` $\rightarrow$ `Term + Term`，再展开 `Term` $\rightarrow$ `number`，然后才去看token `3` 来匹配。永远是先展开规则，再看token。

	int	identifier	=	number	+	;
Statement	<code>$\rightarrow$ Type Ident = Expr ;</code>	—	—	—	—	—
Type	<code>$\rightarrow$ int</code>	—	—	—	—	—
Expression	—	—	—	<code>$\rightarrow$ Term + Term</code>	—	—
Term	—	—	—	<code>$\rightarrow$ number</code>	—	—

这个偷看（lookahead）具体怎么实现，是根据parse table。LL(1) parser 本质上是查表的。Parse table是一个二维表，行是当前的non-terminal，列是下一个token（lookahead）。比如刚才那张表，parser当前在Statement，偷看到 `int`，查表直接告诉你用 `Statement` $\rightarrow$ `Type Identifier = Expression ;`。后来走到Expression，偷看到 `number`，查表告诉你用 `Expression` $\rightarrow$ `Term + Term`。每个格子最多一条production rule，查到就走，不需要回溯，不需要猜。那些空格子呢？比如你在Statement却偷看到 `number`，查表查到空，就会报语法错误。

但如果Expression有两条production rule：`Expression` $\rightarrow$ `Term` 和 `Expression` $\rightarrow$ `Term + Term`，两条都以Term开头，而Term展开后第一个token都是 `number`。那 (Expression, `number`) 这个格子里就有两条entry，这叫conflict。表不知道选哪个。表不再deterministic了。LL（1）也就不知道选哪个。LL(1)没办法parse了。

解决方式有两个：1) 用left factoring，这个视频就不细说了。2) 别用LL（k），用LR（k）

LR parser是bottom-up parser。从token开始，逐步往上归约成更高层的语法结构。
LR parser靠一个stack来工作。它有两种操作：shift（把下一个token压到stack上）和reduce（把stack顶上的一串符号根据production rule归约成一个non-terminal）。

还是 `int x = 3 + 5;`：

1. 看到 `int`，shift, stack: [`int`]
2. `int` 匹配 `Type` $\rightarrow$ `int`，reduce, stack: [`Type`]
3. 看到 `x`，shift, stack: [`Type`, `x`]
4. 看到=，shift, stack: [`Type`, `x`, `=`]
5. 看到 `3`，shift, stack: [`Type`, `x`, `=`, `3`]
6. `3` 是 number，匹配 `Term` $\rightarrow$ `number`，reduce, stack: [`Type`, `x`, `=`, `Term`]

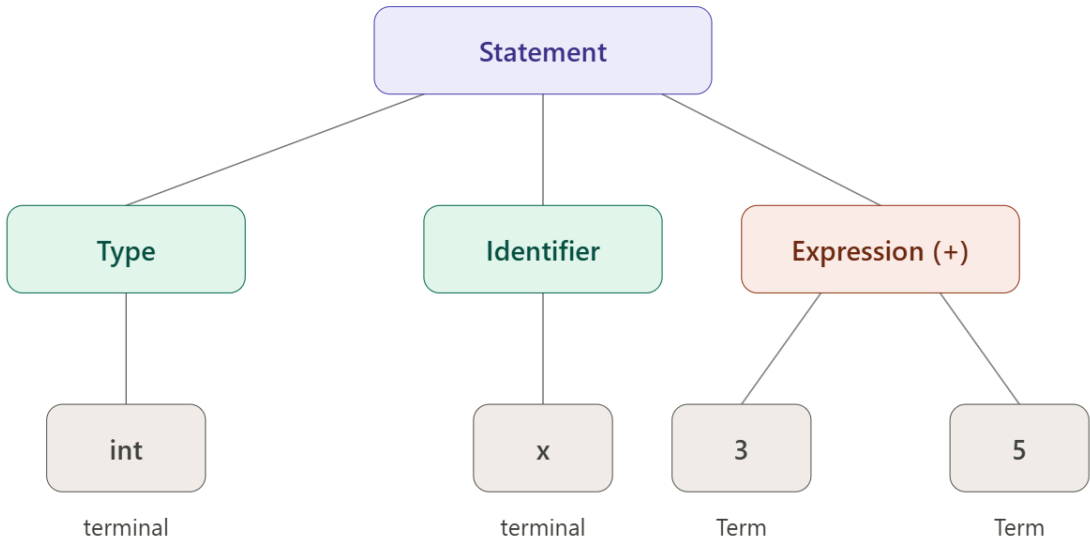
- 7. 看到 + , shift, stack: [Type, x, =, Term, +]
 - 8. 看到 5 , shift, stack: [Type, x, =, Term, +, 5]
 - 9. 5 是 number, reduce, stack: [Type, x, =, Term, +, Term]
 - 10. Term + Term 匹配 Expression → Term + Term , reduce, stack: [Type, x, =, Expression]
 - 11. 看到 ; , shift, stack: [Type, x, =, Expression, ;]
 - 12. 整个 stack 匹配 Statement → Type Identifier = Expression ; , reduce, stack: [Statement] , 完成。
- 注意关键区别：在第6步，LR parser把 3 归约成 Term 之后，它不需要立刻决定 Expression 长什么样。它继续 shift，看到 +，再看到 5，归约成 Term，**这时候**stack 顶上有 Term + Term，才归约成 Expression。决策是推迟到信息足够了才做的。

这就是为什么刚才 LL(1) 搞不定的 Expression → Term | Term + Term，LR 能搞定。LR parser 先归约出一个 Term，再看下一个 token 是 + 还是 ;，就知道该归约成哪条了。

但LR也有conflict。当stack顶上的符号同时能shift也能reduce的时候，叫 **shift-reduce conflict**。当stack顶上的符号同时匹配两条不同的 production rule可以reduce的时候，叫 **reduce-reduce conflict**。碰到这些，LR也不能parse。

gcc和Clang呢，用的是手写的recursive descent parser，既不是LL也不是LR，但是属于top-down。可以在任何地方加任意的lookahead和特殊处理，非常灵活和强大。但是，代价就是巨大的工程量。（其实gcc4.1之前用的是bison，LALR parser）比较热门的DSL parser还有ANTLR，基于LL但是是ALL(*)，不限制 lookahead 数量。

好。通过漫长的parser，我们最终有了Abstract Syntax Tree（AST）。



接下来的semantic analysis（语义分析）会遍历这棵AST，做类型检查（Type Checking）和作用域解析（Scope Resolution）之类的事情。比如 `3 + 5` 两边都是int，没问题。但如果你写 `int x = 3 + "hello";`，parser照样能建出AST（语法上没毛病）但语义分析走到加法节点就会发现int和string不能相加，从而报错。

又比如你用了个没声明过的变量 `y`，语义分析会发现当前作用域(scope)里找不到 `y`，报 `undeclared variable` 错误。

语义分析之后，便是中间表示生成（IR Generation）。做法就是遍历AST，每走到一个节点就生成对应的IR指令。比如走到 **Statement** 节点，看到 **Type** 是 int、**Identifier** 是 x，生成 `alloca i32` 来分配栈空间；走到 **Expression(+)** 节点，看到两个子节点 3 和 5，生成 `add i32 3, 5`；然后生成 `store` 把加法结果存进 x。就这样，AST 上的每个节点都对应一条或几条 IR 指令。注意，这里%x和%1都是寄存器，并且是SSA形式（Single Static Assignment），就是每个虚拟寄存器只能被赋值一次。比如不能有这种东西。到这一步，高层的 `for` 循环、`if-else` 都被拆成了带标签的跳转和基本块（basic block）前端到此结束，接下来就是LLVM的中端范畴了，比如IR优化。

```
%x = alloca i32 ← 来自 Type(int) + Identifier(x)
%1 = add i32 3, 5 ← 来自 Expression(+) → 3, 5
store i32 %1, ptr %x ← 把加法结果存进 x
```

让我们看以下C，经过IR生成，会变成

```
int f() {
    int x;
    x = 4;
    x = 5;
    return x;
}
```

```
define i32 @f() {
    %x = alloca i32
    store i32 4, ptr %x
    store i32 5, ptr %x
    %1 = load i32, ptr %x
    ret i32 %1
}
```

可以看到这里有一个alloca，就是在stack上分配空间。

每次写需要 `store`，读需要 `load`。如果这直接变成最终assembly，每次读写变量都要经过内存，可想而知这成本很高。我们希望把变量存在寄存器里。IR里面的寄存器都是虚拟寄存器。

所以mem2reg，作为IR优化的必备品，负责消除alloca。它分析所有的 `alloca / load / store`，搞清楚数据流关系，把栈操作提升成干净的 SSA 寄存器。

可以看到经过mem2reg，这段IR变成这样：

```
define i32 @f() {
    %x.1 = i32 4 ; 第一次赋值 x = 4
    %x.2 = i32 5 ; 第二次赋值 x = 5
    ret i32 %x.2
}
```

源码里同一个 `x`，现在变成了 `%x.1` 和 `%x.2`。每个值只赋值一次，这就是 SSA。

接下来，就是优化。常见的有constant propagation,

由于我们的例子过于简单，适用的只有dead code elimination。可以看到，`%x.1` 没有被用到。那它可以消失了。`x.2`也没有存在的必要。所以代码只剩下一行。

```
define i32 @f() {
    ret i32 5
}
```

对于其他的编译器优化例子，可以自行查看。非常有趣。

其实我把这些优化步骤拆开了，但实际上compiler只需要遍历一遍就能同时应用很多个优化。

下一步就是编译器后端，机器码生成。

这里GCC和LLVM走了不同的路。

GCC的后端先把IR变成汇编文本（.s 文件），然后调用一个外部的assembler（GNU as）把汇编变成机器码（.o 目标文件）。

gcc的后端做三件事：指令选择（IR 的 `add` 在 x86 上变成 `addl`）、寄存器分配（虚拟寄存器变成 `%eax`、`%ebx` 这些物理寄存器）、指令调度（重排指令顺序来隐藏流水线延迟）。最后吐出一个 `.s` 文本文件。`assembler` 就是查表，把每条指令翻译成对应的二进制。

这中间多了一步文本序列化和反序列化——IR → 汇编文本 → 再parse一遍 → 机器码。

LLVM的开发者觉得这一步是浪费，所以LLVM内置了一个integrated assembler。

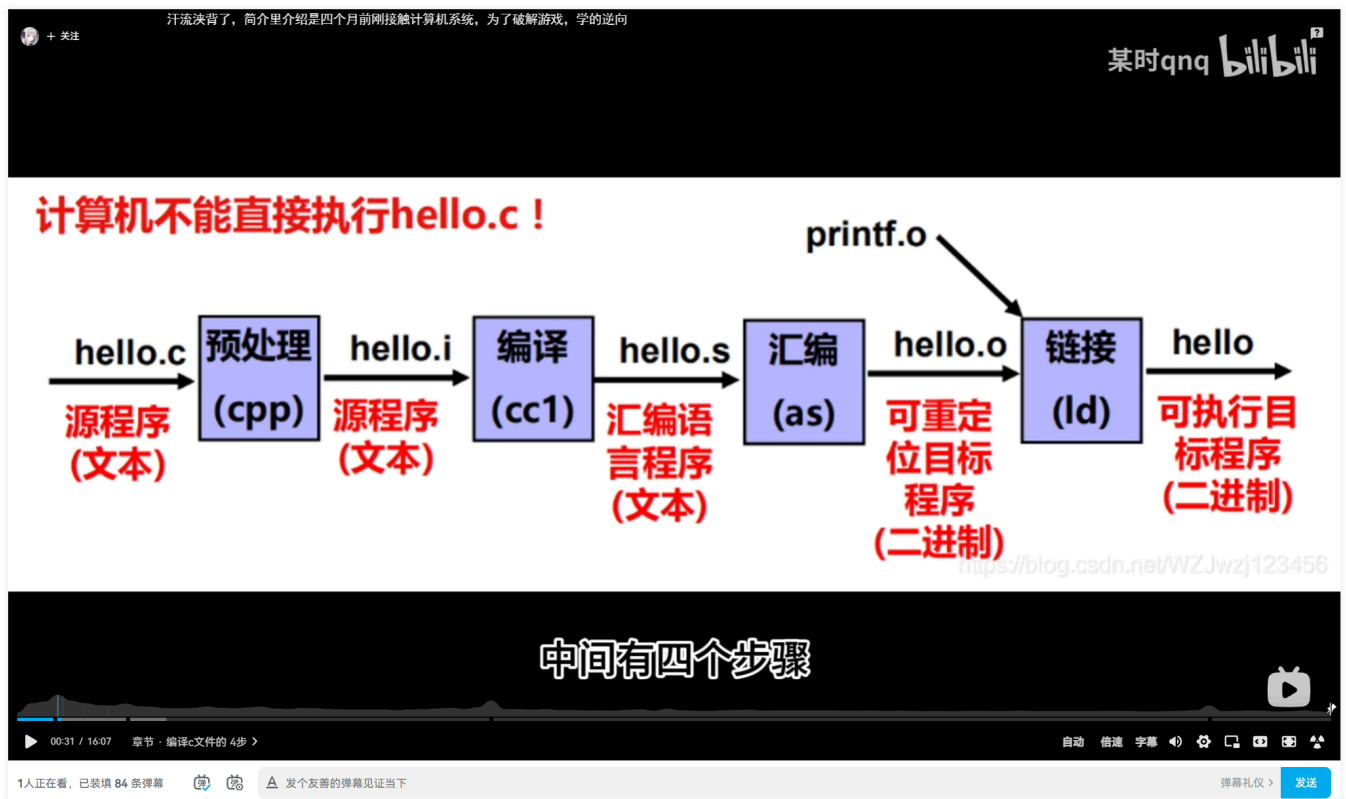
LLVM 后端做的前几步一样——指令选择、寄存器分配、指令调度。但到最后一步，LLVM 有一个叫 **MC layer (Machine Code layer)** 的东西。MC layer 直接在内存里把指令编码成二进制字节，不经过汇编文本。同样是查表编码，只是省掉了“写成文本再 parse 回来”这个来回。

你可以用 `clang -S` 强制让MC layer吐出汇编来看，但默认行为是IR直接变成 `.o` 文件里的机器码。

至此我们拆开了编译型语言的黑盒子。现在你看这张图一定更舒适了。

逆向工程快速入门

6.1万 84 2024-03-11 01:51:05 未经授权，禁止转载



我们终于可以看解释型语言是怎么运作的了，来看 CPython。

CPython 看到一段 `.py` 文件，会走类似编译器前端的步骤：lexer、parser、AST、最后生成 bytecode。生成的 bytecode 会缓存在 `.pyc` 文件里。

不过有一点和 C 不一样——CPython 不做完整的语义分析，尤其是类型检查。因为 Python 是动态类型语言，变量的类型是运行时才知道的事，编译阶段管不了。

那这份 bytecode，不会被编译成机器码。而是由 CPython 去逐条执行。CPython 本身是用 C 写的，早就编译成了机器码，作为一个 binary 跑在你电脑上。所以当你 `pip install python`，你本质上就是在下载这个 CPython 的 bin。

CPython 内部有一个巨大的 dispatch loop。每看到一条 bytecode 指令，就跳到对应的 C 实现去执行，执行完再跳回来，读下一条。这就是 **Tier 1**。

这里有个关键问题：为什么这个慢？拿 `a + b` 举例。C 编译器在编译时就知道 `a` 和 `b` 都是 int，直接生成一条 `add` 机器指令，完事。但 CPython 看到 `BINARY_OP` 这条 bytecode，得先取出 `a` 的类型——哦，是 int；再取出 `b` 的类型——也是 int；然后查：int + int 该调哪个函数？找到了，调用 `int.__add__`。一条加法，C 是一条机器指令，CPython 可能是几十上百条。这就是 Python 比 C 慢的核心原因。而且还有另一个问题：Tier 1 的 dispatch loop 每条指令都有一次 indirect branch，CPU 的分支预测对这种跳来跳去不太擅长，预测失败就要 pipeline flush，性能损耗很大。

所以 Python 3.13 引入了 **Tier 2**，来解决这两个问题。

Tier 2 的思路是：找到反复执行的热点代码，把它"展开"成一个线性的 uop trace 数组，顺序执行，不回 dispatch loop。

当代码在 Tier 1 的 dispatch loop 里跑的时候，每条 bytecode 都被计数，这条指令执行了多少次。某条指令执行次数超过阈值，Tier 1 会先把它特化。比如 `BINARY_OP` 发现每次来的都是 int，就把自己替换成 `BINARY_OP_ADD_INT`。这还是在 Tier 1 里跑，只是那条指令变专了。执行这个 opcode 就不用再做类型验证了，可以减少一些执行的代码（稍微快一点）。

继续跑，计数继续涨。超过更高的阈值，CPython 说：好，这段值得建 trace 了。于是开始录制——跟着 Tier 1 执行的路径，把经过的 uop 一条条记下来，拼成数组。trace 建好之后，入口处那条 bytecode 被打上标记。下次 Tier 1 执行到这里，看到标记，不再走 dispatch loop，直接跳进 trace 数组，从左到右顺序执行，执行完再回到 Tier 1。

比如循环里反复跑这几条：

```
LOAD_FAST a
LOAD_FAST b
BINARY_OP +
STORE_FAST x
```

Tier 2 把它们拼成一条 trace：

```
[ GUARD_TYPE_INT(a) | GUARD_TYPE_INT(b) | BINARY_OP_ADD_INT | STORE_FAST_x ]
```

注意开头多了两个 `GUARD`，这是关键。
Guard 解决两个问题：怎么进，怎么出。
Trace 开头的 guard 是入口条件。每次执行到这里，先检查 `a` 是不是 int，`b` 是不是 int。如果是，放行，进入 trace，直接顺序走完，不做任何类型分发。
如果 guard 失败——比如某次循环 `a` 突然变成了 float——trace 直接退回 Tier 1 的 dispatch loop 处理。

那我们现在把跳转可以用 uop trace 来消灭掉。
能不能更快一点？可以的。用 JIT (Just In Time) 编译。
本质就是，我们既然有了这一段被反复执行的 uop trace，我们更进一步把这个 trace 编译成机器码，存到内存里。下次要进这个 trace 执行，就不再是 CPython 去读 trace 数组，而是直接跳进那块机器码，CPU 全速跑。
这就是 JIT，在代码跑的过程中找到热点代码（被反复执行的代码）然后编译成机器码存内存里。

那这期视频就到这里啦。我介绍了很多传统编译器的做法。下一个视频会讲 ML 编译器。